Compressing ACAS-Xu Lookup Tables with Binary Decision Diagrams

Martin Boniol[1,2][0009−0005−0544−6778], Julien Brunel[2][0009−0004−3639−6681], Jean-Baptiste Chaudron[1][0000−0002−2142−1336], Christophe Garion[1][0000−0002−4467−2939], and Xavier Thirioux[1][0009−0002−1126−6835]

[1] Université de Toulouse, ISAE SUPAERO, Toulouse, France
[2] Université de Toulouse, DTIS, ONERA, Toulouse, France

This is a preprint of a paper accepted for publication at NFM 2026.
This version has not undergone final publisher typesetting.
The final authenticated version will be published by Springer in LNCS.

Abstract. The Airborne Collision Avoidance System Xu (ACAS-Xu) relies on large certified Look-Up Tables (LUTs) that encode the exact decision logic used in operation. Neural-network-based approximations have been proposed to reduce memory requirements, but they inherently introduce approximation errors and complicate formal verification. This paper presents a symbolic compression approach based on Binary Decision Diagrams (BDDs) that preserves the exact semantics of the ACAS-Xu LUTs. The resulting representation is canonical, deterministic, and fully equivalent to the original tables, enabling sound and exact reasoning over the complete decision logic. By expressing both the system behavior and domain-specific operational properties within a common Boolean framework, verification reduces to efficient BDD operations and emptiness checks, with precise counterexamples generated when properties are violated. We demonstrate that the proposed BDD-based representation significantly reduces memory usage, achieves predictable and low-latency execution, and can be deployed on embedded platforms. These results highlight BDDs as a compelling alternative for exact, verifiable, and embedded deployment of ACAS-Xu decision logic.

Keywords: Binary Decision Diagram · Formal Verification · ACAS-Xu · Neural Network · Embedded System · UAV.

1 Introduction

Ensuring safe separation between aircraft is a critical challenge in increasingly dense airspace. The ACAS-X addresses this challenge through a family of collision avoidance logic, including ACAS-Xu, a variant designed specifically for Unmanned Aerial Vehicles (UAVs) and supporting both vertical and horizontal maneuvers [17,16].

The ACAS-Xu decision logic is implemented using large LUTs. While LUTs guarantee deterministic behavior, their memory footprint poses a serious obstacle to deployment on UAV platforms. In particular, the horizontal avoidance

tables alone require around 5 gigabytes of storage, challenging the capabilities of typical onboard avionics [24]. Moreover, the latency of accessing such large tables compromises real-time performance on resource-constrained platforms.

To alleviate this issue, prior work has explored compressed representations of the ACAS-Xu logic, most notably using Neural Networks (NN) [20]. While such approaches significantly reduce memory requirements, they rely on continuous function approximation of discrete decision boundaries and therefore do not preserve the exact semantics of the original LUTs, raising concerns for safety-critical deployment [4,13].

In this work, we propose a symbolic compression technique for ACAS-Xu based on Binary Decision Diagrams (BDDs). Our BDDs provide a canonical and exact symbolic representation of discrete decision functions, enabling substantial reductions in memory footprint while preserving the precise behavior of the original LUTs [7,9] eliminating the approximation errors inherent in NNs approaches. By encoding both the decision logic and operational properties within a unified Boolean framework, we enable efficient formal verification through direct BDD operations, providing exact counterexamples when properties are violated. Then we demonstrate that this representation significantly reduces memory footprint and supports real-time execution on resource-constrained platforms, including both CPU and FPGA-accelerated implementations.

The rest of this paper is organized as follows: Section 2 introduces the ACAS-Xu system and Section 3 presents a compression strategy using NNs. Section 4 details the BDD-based compression of the ACAS-Xu LUTs. In Section 5 we present a strategy to verify our BDDs. Section 6 details the comparison between NNs, BDDs and LUTs through experimental evaluation of each implementations on embedded systems. Finally, Section 7 concludes and outlines perspectives for future work.

2 ACAS-Xu presentation

2.1 Decision logic and advisories

ACAS-Xu is the collision avoidance system designed for unmanned aircraft within the ACAS-X family [17]. It determines whether an avoidance maneuver is required by evaluating the relative geometry between the ownship and surrounding traffic. The system operates on a six-dimensional state vector derived from sensor measurements and relative kinematics, as illustrated in Figure 1. This state is defined by the following parameters: τ (time until loss of vertical separation), ρ (distance between the ownship and the intruder), θ (angle to the intruder relative to the ownship's heading direction), ψ (intruder's heading angle relative to the ownship's heading), v_{own} (ownship speed), v_{int} (intruder speed).

Based on this state representation and and the previous advisory issued (a_{prev}), ACAS-Xu selects an appropriate avoidance resolution advisory. For horizontal conflict resolution, the system can issue one of five possible advisories : Clear of Conflict (COC), Weak Left (WL), Weak Right (WR), Strong Left (SL) and Strong Right (SR)

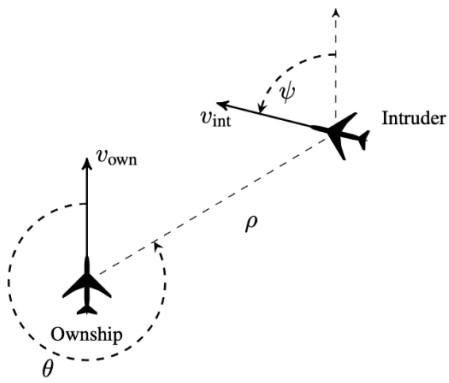

Fig. 1: ACAS-Xu geometry (taken from [22])

2.2 LUTs as the Core Decision Representation

The ACAS-Xu decision logic relies on several structural features that characterize how the system evaluates potential collision scenarios. When an advisory must be issued, the system first acquires the current encounter state, which describes the relative geometry and kinematics of the ownship and the intruder. This continuous state is then discretized according to the predefined ACAS-Xu quantization grids, and the corresponding LUT entry is selected by mapping the state to its nearest discretized point [16,17].

For each discretized encounter state the LUT stores the expected cost (also called score) associated with every possible advisory, as computed offline using Markov Decision Processes (MDPs) [23]. These advisory costs are all returned by the LUTs but the advisory minimizing this expected cost is selected to be the advisory provided to the pilot. Each LUT covers nearly 90 million discretized encounter states, representing a comprehensive set of possible relative configurations under varying geometries, speeds, and sensor uncertainties. In addition, the system maintains five distinct LUTs, one for each value of the previously issued advisory (a_{prev}). The global architecture is shown in Figure 2.

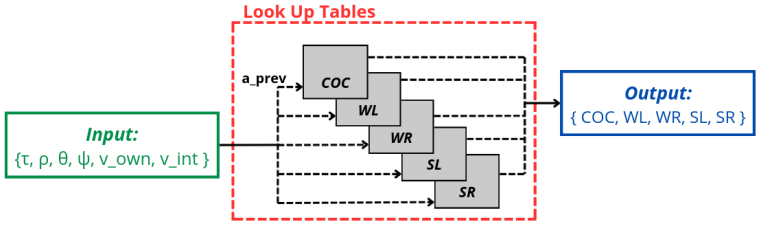

Fig. 2: ACAS-Xu Architecture

3 Neural Network–based compression and its limitations

3.1 The Reluplex NNs

Neural networks have been widely investigated as a way to compress the ACAS-Xu LUTs, primarily due to their ability to approximate high-dimensional decision functions with a substantially smaller memory footprint [19].

We report here experimental results obtained with the ACAS-Xu networks originally introduced by Julian et al [20]. These models constitute the most widely used reference implementation in the literature, and evaluating them offers a representative view of the challenges encountered when approximating the ACAS-Xu decision logic with NNs.

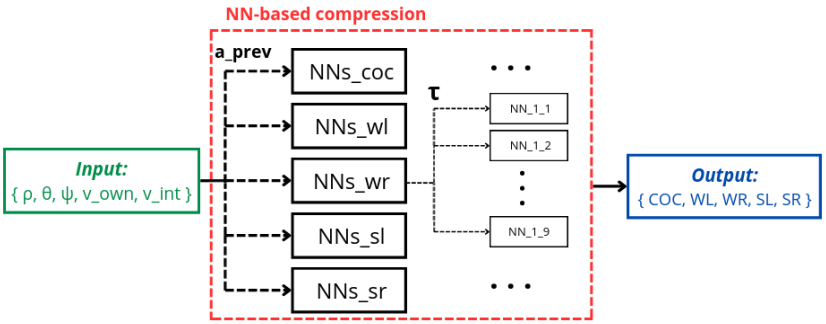

Fig. 3: ACAS-Xu Architecture with NN

The ACAS-Xu advisory decision process is implemented through an array of 45 distinct Deep Neural Networks, each trained to operate within a specific region of the state space. They are obtained by partitioning each of the 5 original a_{prev}-dependent LUTs into 9 separate models, each corresponding to a discrete value of the τ dimension (time to loss of vertical separation). The architecture overview is shown in Figure 3.

3.2 NNs analysis

When trained on LUT data, these models can often reproduce the advisory logic with an average accuracy [21]. Several structural limitations affect their suitability for safety-critical decision systems. NNs do not provide deterministic guarantees on their outputs, and small variations in the input space may lead to inconsistent or incorrect advisories [8]. These failures typically occur in localized regions of the state space, where decision boundaries are sensitive, sparse, or difficult to approximate accurately [21,19].

To assess the reliability of these networks, we evaluate a subset of five NNs, arbitrarily selected as the first network associated with each LUT. The evaluation

is performed on two targeted test sets extracted from the original LUTs: the COC regions (states for which the LUT returns the advisory Clear of Conflict) and the Non-COC regions (states requiring an avoidance maneuver like WL, WR, SL or SR). For each test set, the evaluation consists in a point-wise comparison between the outputs produced by the neural networks and the corresponding LUT advisories over all states contained in the set.

NN	**LUT Points (COC)**	**LUT Points (Non-COC)**
NN_{1_1}	97.76%	57.44%
NN_{2_1}	96.58%	69.95%
NN_{3_1}	92.30%	68.53%
NN_{4_1}	98.54%	59.58%
NN_{5_1}	98.22%	59.87%

Across all tested networks, the results reveal a consistent and marked discrepancy between performance on COC and Non-COC regions. On COC states, the networks achieve very high conformity rates, typically above 96%, indicating that the models are able to accurately learn and reproduce the dominant "no-action" regions of the LUTs. This behavior is expected, as COC regions occupy a large and relatively homogeneous portion of the state space, making them easier to approximate using standard supervised learning techniques. In contrast, performance on Non-COC regions is significantly lower. Conformity rates for states requiring an avoidance maneuver range from approximately 57% to 70%, depending on the network. These results highlight a systematic weakness of the neural models precisely in the regions where correct behavior is most critical for safety. Non-COC regions correspond to narrower, more complex, and highly structured decision boundaries, where small changes in the state can trigger different advisories.

The observed performance gap between COC and Non-COC regions highlights a central challenge of neural-network-based LUT compression for ACAS-Xu. These results indicate that neural approximations alone struggle to provide uniform correctness guarantees across the entire operational domain, and that average accuracy metrics are insufficient to characterize their reliability in safety-critical settings. This limitation has motivated the exploration of hybrid solutions that aim to combine the efficiency of neural networks with the exactness of the original LUTs. In particular, several approaches propose complementing neural decision logic with selected portions of the LUT in regions where the network is known to be unreliable [13,10]. By reverting to the exact LUT advisory in these cases, such mechanisms, often referred to as Safety Nets, seek to retain the compactness of neural representations while mitigating their most critical failure modes. Nevertheless, maintaining such LUT fragments partially offsets the benefits of compression and adds complexity to the overall architecture.

4 BDD for representing ACAS-Xu logic

The limitations of NNs highlight the need for alternative representations that remain compact while preserving the exact behavior of the original LUTs. In

this context, symbolic data structures offer a promising direction. Among them, BDDs provide a canonical and deterministic encoding of discrete functions, making them well suited for representing the ACAS-Xu decision logic without approximation. Building on these properties, this section introduces our BDD-based compression approach and discusses how it addresses the constraints associated with LUT storage on UAV platforms.

4.1 Presentation and illustrative example of a BDD

BDDs are symbolic data structures used to represent Boolean or discrete functions in a compact and canonical form [7]. A BDD encodes a function as a directed acyclic graph in which each internal node corresponds to the evaluation of a variable, and each terminal node corresponds to an output value. Under a fixed variable ordering, the BDD representation is unique, which provides strong guarantees on determinism and simplifies equivalence checking [7]. BDDs are widely used in formal verification, model checking, and hardware synthesis due to their ability to represent large state spaces efficiently [9,25].

To illustrate the principle of BDDs, we consider the Boolean function $f(x, y, z) = (x \wedge y) \vee (y \wedge z)$ and compare its representation as a decision tree and as BDDs in Figure 4.

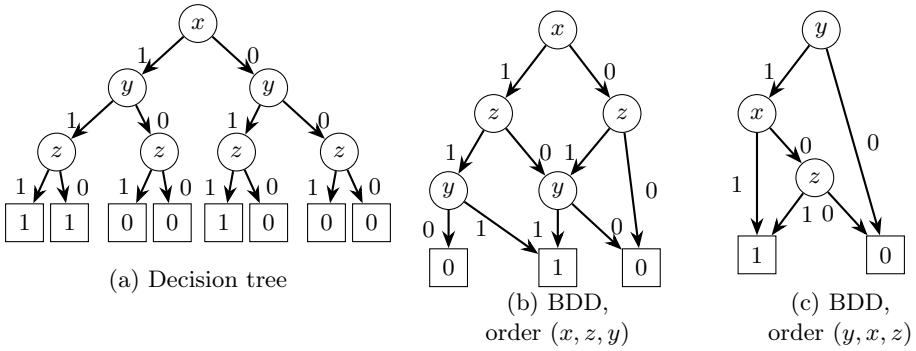

Fig. 4: Decision tree and BDD representations of $f(x, y, z)$.

The decision tree in Figure 4(a) evaluates function by testing variables sequentially along a root-to-leaf path. For the function $f(x, y, z)$ this results in multiple distinct subtrees rooted at the variable z, which are reached under different assignments of the preceding variables x and y. A BDD eliminates this redundancy by explicitly sharing equivalent substructures. Under a fixed variable ordering, identical subfunctions are represented by a single node and reused across multiple evaluation paths. This effect is illustrated in Figure 4(b), where the BDD constructed with a suboptimal variable ordering exhibits limited

sharing and therefore remains relatively large. The importance of the variable ordering becomes apparent when comparing this representation with the BDD obtained under a more suitable ordering. In Figure 4(c), placing y early in the ordering exposes common decision patterns, allowing the corresponding subgraphs to be fully shared. This ability to factorize common decision patterns is essential when representing large discrete decision functions, such as the ACAS-Xu advisory logic, provided that an appropriate variable ordering is chosen.

4.2 BDD construction procedure

In the context of ACAS-Xu, several characteristics of the decision logic make BDDs particularly well suited. The LUTs map discretized encounter states to a finite set of advisories, forming a deterministic function over a structured and fully discrete domain. This setting aligns naturally with BDD representations, which can exploit redundancies and common patterns across the input space to drastically reduce the size of the encoded function. In our construction, the BDDs capture for now only the final advisory decision associated with each state, whereas the original LUTs additionally store the underlying cost or score values used during offline optimization.

Fundamentally, the decision-making process of the ACAS-Xu is a geometric partitioning problem: large regions of the state space share the same optimal advisory, leading to significant data redundancy. By representing these LUTs as BDDs, we can exploit this underlying structure. Moreover, BDDs preserve the exact behavior of the original LUTs: no approximation is introduced, and every state maps deterministically to the same advisory as in the reference tables. While alternative structures are possible, our design choices were specifically guided by the need to demonstrate the practical suitability and efficiency of a BDD-based approach for real-time execution on resource-constrained embedded platforms. Consequently, we develop in this paper a architecture composed of five BDDs, each representing one of the system's LUTs based on the previous advisory (a_{prev}), cf. Figure 5. A natural alternative would be to encode a_{prev} as an additional input within a single, unified BDD to maximize node sharing across all tables. However, we deliberately chose a partitioned architecture for two main reasons because we wanted to maintain strict alignment with the legacy ACAS-Xu operational logic, which is natively structured as independent tables. This facilitates the traceability and certification of the compressed representation.

In our approach, BDDs are used not merely as a symbolic encoding of the ACAS-Xu decision logic, but explicitly as a classifier over the discretized state space. Instead of representing a single Boolean function, the diagram encodes the advisory decision by associating a Boolean predicate to each of the five possible actions. Each predicate characterizes precisely the set of states for which the LUT prescribes that advisory. This structure allows the system to determine the prescribed action by evaluating which advisory predicate evaluates to `true` for a given input state. Because the advisory sets are provably disjoint, exactly one predicate will hold for any valid encounter state, providing a deterministic and unambiguous classification mechanism. Viewed this way, the BDD functions

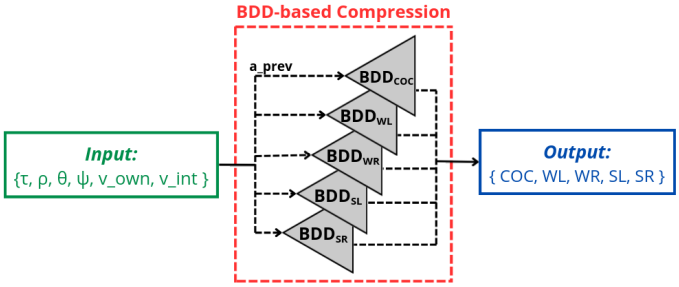

Fig. 5: ACAS-Xu Architecture with BDDs

are an exact, symbolic multi-class classifier whose decision boundaries coincide perfectly with those of the original ACAS-Xu LUTs.

To encode the ACAS-Xu LUTs as BDDs, each LUT entry is first identified by a discrete state tuple $(\tau, \rho, \theta, \psi, v_{own}, v_{int})$ where each component corresponds to a predefined quantization level of the associated state variable. We mapped therefore each state to a tuple of integer indices $(\tau^i, \rho^i, \theta^i, \psi^i, v^i_{own}, v^i_{int})$ where each index uniquely identifies the corresponding table variable along that dimension. Then we encoded these integer indices in binary to form the Boolean input variables of the BDD. For each dimension, we select the minimum number of bits required to represent all possible index values. Specifically, the three parameters τ, v_{own} and v_{int} are encoded each on four bits, in order to cover the 10 possible values of τ, the 12 values of v_{own}, and the 12 values of v_{int}. Similarly, six bits are used to encode each of the three last parameters ρ, θ and ψ, in order to cover the 41 values of ρ, the 39 values of θ, and the 39 values of ψ. This encoding yields a compact Boolean representation of the discrete ACAS-Xu state space, suitable for symbolic manipulation using BDDs.

Because the ACAS-Xu decision problem is fundamentally geometric, the structure of the advisory logic exhibits strong spatial regularities. Advisories tend to form contiguous regions in the discretized state space, corresponding to avoidance maneuvers that vary smoothly with respect to the state parameters. As a result, it is unlikely for the recommended action to change abruptly between two successive index values along any single dimension. We have already emphasized the critical role of variable ordering in achieving compact BDD representations. In the same spirit, the choice of how discrete indices are encoded at the bit level has a direct impact on the amount of structural sharing that can be exploited during BDD construction. To promote compactness, we therefore encode all state indices using fixed-size Gray code representations. This encoding ensures that consecutive index values differ by only a single bit, thereby preserving the geometric proximity of neighboring states in the Boolean domain [14]. By limiting the number of bit flips between adjacent discrete states, Gray encoding should increase the likelihood that nearby regions of the state space share common subgraphs in the BDD. In practice, we consistently observed a significant reduction in the total number of BDD nodes when using Gray code compared

to a standard binary representation. This encoding effectively facilitates the merging of equivalent decision structures, leading to more compact and efficient diagrams.

Each discretized state is then mapped to a single Boolean cube, defined as the conjunction of all bit literals corresponding to that state. This cube evaluates to `true` for that exact state and for no other. For each advisory, we collect all corresponding cubes and merge them through disjunction to form one BDD root per advisory. This produces five separate BDDs, one for each LUT action, that exactly encode the content of the table, since each LUT entry corresponds to a unique cube in its associated root.

4.3 BDD implementation

Our construction relies on the CUDD library [28], accessed through the `dd.cudd` Python wrapper [18]. CUDD provides an optimized BDD manager that supports fast Boolean operations, unique-table-based node canonicalization, garbage collection, and dynamic variable reordering. These capabilities are essential as the ACAS-Xu LUTs contain millions of entries.

The construction of the BDDs was performed incrementally. All Boolean variables are declared at initialization and assigned an initial fixed ordering that directly reflects the structure of the ACAS-Xu LUTs. This ordering, while convenient and intuitive, is an arbitrary design choice. Specifically, variables are ordered dimension by dimension and bit by bit, following the natural LUT layout, where index 0 denotes the most significant bit of each encoded parameter:

$$\begin{bmatrix} \tau^0, \tau^1, \tau^2, \tau^3, \rho^0, \rho^1, \rho^2, \rho^3, \rho^4, \rho^5, \theta^0, \theta^1, \theta^2, \theta^3, \theta^4, \theta^5, \psi^0, \psi^1, \psi^2, \psi^3, \psi^4, \psi^5, \\ v^0_{own}, v^1_{own}, v^2_{own}, v^3_{own}, v^0_{intr}, v^1_{intr}, v^2_{intr}, v^3_{intr} \end{bmatrix}$$

The LUT files are processed in chunks to limit peak memory usage. Each chunk is translated into a set of Boolean cubes, merged into a temporary BDD, and then incorporated into the corresponding advisory root. During construction, we periodically invoke CUDD's dynamic variable reordering mechanisms [27], which aim to identify a variable order that minimizes the overall diagram size. The impact of variable reordering is known to be substantial. For example, when constructing the BDD corresponding to the LUT with previous advisory SR, the initial ordering yields a diagram containing $902,180$ nodes. After applying CUDD's reordering, the diagram size is reduced to $483,638$ nodes, representing a reduction of nearly 46%. The resulting variable order is:

$$\begin{bmatrix} \rho^0, \rho^1, \psi^1, \psi^0, \theta^0, \theta^1, v^1_{int}, v^0_{int}, v^0_{own}, v^1_{own}, \theta^2, \psi^2, \psi^3, \theta^3, \theta^4, \psi^4, \theta^5, \psi^5, \\ v^2_{int}, v^2_{own}, v^3_{int}, v^3_{own}, \rho^4, \rho^5, \rho^3, \tau^3, \rho^2, \tau^2, \tau^0, \tau^1 \end{bmatrix}$$

An analysis of the variable ordering given by CUDD provides additional insight into the structure of the ACAS-Xu decision logic. In particular, the bits associated with the variable τ are consistently pushed to the end of the ordering, indicating that τ has a limited influence on the advisory outcome compared to the other state variables. By contrast, the remaining dimensions are ordered such

that their most significant bits appear early in the diagram. This observation suggests that the partitioning of neural networks along the τ dimension, as performed in [20], may not be the most relevant decomposition strategy. Finally, when initializing CUDD's dynamic reordering with the ordering obtained here, the algorithm converges to an almost identical configuration, which indicates that this ordering seems closed to a local optimum.

Each advisory root is dumped independently during construction, ensuring debuggability and reproducibility. And then final diagrams are stored and loaded using the DDDMP format provided by CUDD.

4.4 Partitioning check and root reduction

Once the five advisory BDDs are constructed, we verify that they form a partition of the discretized state space. First, we verify that they are pairwise disjoint by computing the conjunction of every pair of BDDs. Since BDDs are a canonical representation, the fact that each conjunction evaluates to the constant `False` constitutes a formal proof that the advisory sets are mutually exclusive across the entire state space, as expected from the LUT semantics. Second, we verify that their union covers the entire state space: $root_1 \vee root_2 \vee root_3 \vee root_4 \vee root_5 = \top$

Together, these two properties ensure a proper partitioning: every discretized state belongs to exactly one advisory set. This partitioning property has an important practical consequence. Since the advisory sets are mutually exclusive and collectively exhaustive, one of the BDDs can be represented implicitly as the complement of the union of the others. In particular, we remove the largest BDD and define it as $root_5 = \neg(root_1 \vee root_2 \vee root_3 \vee root_4)$

When the eliminated BDD is substantially larger than the others, this substitution leads to a noticeably more compact overall representation. In our construction, we therefore retain only four explicit BDDs and rely on this complement-based representation for the fifth advisory.

4.5 Final single-root BDD

The advisory-specific BDDs are assembled into a single, deployable diagram by introducing two selector variables, $bdd0$ and $bdd1$. Each advisory is associated with a fixed pattern over these two variables. Figure 6 is an example of our BDD encoding, in the context of the LUT corresponding to the previous action SR.

For each retained advisory, we construct a guarded subgraph of the form $sel_a \wedge root_a$ and combine all guarded subgraphs through disjunction:

$$global_{root} = \bigvee_{a \in Advisories} (sel_a \wedge root_a)$$

The two selector variables $bdd0$ and $bdd1$ are introduced as additional Boolean inputs and are placed at the very top of the global BDD. Their position in the variable ordering is fixed and they are explicitly excluded from the dynamic reordering process. As a result, every evaluation of the global diagram begins by

$$
\begin{aligned}
sel_{COC} &= bdd1 \wedge bdd0 \\
sel_{SL} &= \neg bdd1 \wedge bdd0 \\
sel_{WR} &= bdd1 \wedge \neg bdd0 \\
sel_{SR} &= \neg bdd1 \wedge \neg bdd0
\end{aligned}
$$

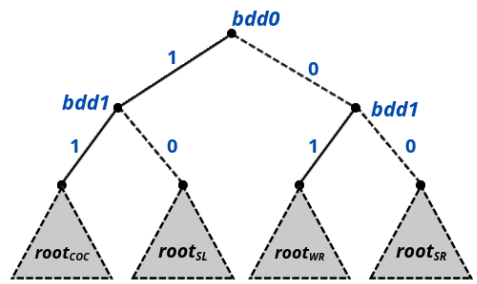

Fig. 6: Final Single-Root BDD_{SR} illustration (a_{prev} = SR)

assigning values to these two selector variables, thereby deterministically selecting the advisory of interest before any state-dependent decision logic is traversed. By fixing the selector variables at the root of the diagram, the structure of each advisory-specific subgraph remains unchanged and fully isolated from the others. It also allows the independent evaluation of a specific advisory by setting $(bdd1, bdd0)$ to the corresponding binary pattern and evaluating the resulting Boolean function. In theory, placing these selector variables at the very end of the ordering would be beneficial, as it would allow for a single BDD traversal to determine the appropriate decision. However, since BDD optimization tools cannot typically 'force' variables to the bottom of the ordering without significant overhead or loss of efficiency, fixing them at the top remains the most robust way to ensure a predictable and modular traversal during deployment.

Since the advisory roots are disjoint, each input state satisfies exactly one predicate. The fifth advisory is implicitly represented by the negation of the union of the four explicit ones. Let us notice that the four retained advisories may differ from one LUT to another. In the case of the LUT corresponding to the previous action SR, the four retained advisories are COC, SL, WR and SR. We show in Figure 6 the final single-root BDD in the case of $a_{prev} = SR$.

5 Properties verification using BDD

5.1 Verification through domain-specific properties

The BDDs obtained were first validated against the original ACAS-Xu LUTs. As expected, the symbolic representation ensures strict equivalence: *for every discretized state contained in the LUT domain, the advisory returned by the BDD-based decision logic matches the reference LUT output.*

Beyond functional validation, a key advantage of the BDD-based representation is that it naturally enables formal verification of decision logic properties. Because both the system behavior and the verification conditions can be expressed symbolically, property checking reduces to standard boolean reasoning over decision diagrams.

Following the methodology introduced in prior work such as Reluplex-based analyses [22], domain-specific properties can be formalized as constraints over the encounter state and the issued advisory. In our setting, the properties are encoded by two separate BDDs: BDD_{prop}, which characterizes the set of states for which the property's preconditions hold and BDD_R which encodes the expected outcome of the property. The compressed ACAS-Xu logic itself is represented by BDD_{acas}. Property satisfaction can then be checked symbolically by verifying the logical implication: $BDD_{acas} \wedge BDD_{prop} \Rightarrow BDD_R$. This is equivalently tested by checking the emptiness of the following intersection: $BDD_{acas} \wedge BDD_{prop} \wedge \neg BDD_R$. If this intersection evaluates to the empty BDD, the property is formally proven to hold for all states satisfying the constraints. Conversely, if the intersection is non-empty, the satisfying assignments correspond to concrete counterexamples that witness a violation of the property. These counterexamples can be extracted directly from the BDD, providing precise and interpretable diagnosis.

Property 3. If the intruder is directly ahead and is moving towards the ownship, the advisory will not be COC.
Input constraints : $1500.0 \leq \rho \leq 1800.0$, $\psi \geq 3.10$, $v_{own} \geq 980.0$, $v_{int} \geq 960.0$
Desired Output : not COC

For instance, to check such properties we first translate the real-valued constraints into the symbolic domain. This process involves three steps:

- Discretization: Each real-valued interval $[v_{min}, v_{max}]$ is mapped to its corresponding set of indices i_{min}, ... ,i_{max} along its dimension in the original ACAS-Xu lookup tables.
- Boolean Encoding: Each index i is converted into its Gray code representation to maintain structural locality in the BDD.
- Symbolic Construction: For each variable x, we construct a BDD C_x representing the disjunction of all bit-vector cubes corresponding to the valid indices: $C_x = \bigvee_{j=i_{min}}^{i_{max}} \text{Cube}(\text{Gray}(j))$.

The complete precondition BDD is then the conjunction of these individual constraints: $BDD_{prop} = \bigwedge_{x \in \text{Inputs}} C_x$. For Property 3, the input constraints on distance (ρ), angles (θ, ψ), and velocities (v_{own}, v_{int}) are combined into a single BDD_{prop}. The verification then checks if any state in BDD_{prop} leads to a COC advisory by evaluating the intersection $BDD_{acas} \wedge BDD_{prop} \wedge BDD_{COC}$, where BDD_{COC} is the symbolic function corresponding to the COC output (defined by the selectors $bdd0 \wedge bdd1$). Any satisfying assignment in the resulting BDD directly provides a counterexample in the form of a specific LUT state where the property is violated.

The results of our verification experiments are reported on Table 1. These properties were originally proposed in [22] and verified on a small subset of the NNs, but not on all 45 networks representing the complete system without a

Table 1: Verification of domain-specific properties from [22] on BDDs. Property P_8 applies only to scenarios with $a_{prev} = WL$ and can therefore be evaluated only on BDD_{WL}.

P	**BDD$_{\mathbf{COC}}$**	**BDD$_{\mathbf{WL}}$**	**BDD$_{\mathbf{WR}}$**	**BDD$_{\mathbf{SL}}$**	**BDD$_{\mathbf{SR}}$**
$\mathbf{P_1}$	Invalid	Valid	Valid	Invalid	Invalid
$\mathbf{P_2}$	Invalid	Valid	Valid	Invalid	Invalid
$\mathbf{P_3}$	Valid	Valid	Valid	Valid	Valid
$\mathbf{P_4}$	Valid	Valid	Valid	Valid	Valid
$\mathbf{P_5}$	Valid	Valid	Invalid	Valid	Invalid
$\mathbf{P_6}$	Valid	Valid	Valid	Valid	Valid
$\mathbf{P_7}$	Valid	Invalid	Invalid	Valid	Valid
$\mathbf{P_8}$	N/A	Valid	N/A	N/A	N/A
$\mathbf{P_9}$	Valid	Valid	Valid	Valid	Valid
$\mathbf{P_{10}}$	Valid	Valid	Valid	Valid	Valid

clear explanation. Extending the evaluation to the complete ACAS-Xu domain may reveal unforeseen behaviors in certain regions of the state space.

The results reported in Table 1 admit a twofold interpretation. First, for each `Invalid` property, the BDD-based verification procedure produces an explicit counterexample in the form of a concrete discretized encounter state. Because the constructed BDDs are exact symbolic representations of the original ACAS-Xu LUTs, these counterexamples can be directly evaluated against the LUTs themselves. In all cases, the counterexamples identified by the BDD verification process correspond to genuine violations in the reference LUTs. These results confirm the correctness of the BDD encoding and reveal a discrepancy between properties verified on NN approximations and the behavior encoded in the original ACAS-Xu LUTs. Specifically, several domain-specific properties that were verified on a limited set of neural networks using Reluplex [22] do not hold when evaluated against the LUTs. This observation raises a broader question about how such properties should be interpreted and validated. The second concerns the effectiveness of BDDs as a verification substrate and is detailed in 5.2.

5.2 BDDs as an effective verification tool for ACAS-Xu

In contrast to LUT-based verification, which is computationally prohibitive, and NN-based approaches, which require specialized solvers and introduce approximation artifacts, BDDs offer a unique combination of exactness and tractability. Properties can be encoded directly as Boolean constraints and composed with the system BDD using standard logical operations, enabling efficient validation and precise counterexample extraction at the scale of the full decision logic.

Moreover, the symbolic nature of BDDs significantly broadens the class of properties that can be verified. While verification of neural-network approximations typically relies on interval-based abstractions over input parameters, BDDs naturally support relational properties involving multiple state variables.

This makes possible to reason about more expressive operational and safety constraints without modifying the underlying system representation, and supports incremental verification workflows well suited for certification and safety analysis. We provide below an example of such a relational operational property that can be easily verified using the BDD-based framework.

Property 11. If the intruder is approaching the ownship from behind at a lower speed, the system will advise COC.
Input constraints : $v_{int} \leq v_{own}$, 8500.0 $\leq \rho \leq$ 62000.0, -3.1416 $\leq \theta \leq$-3.1416 + 0.01, -0.06 $\leq \psi \leq$ 0.06
Desired Output : COC

The verification of such a property using BDDs reduces to a simple symbolic intersection test. First, the input constraints are encoded as a BDD, denoted BDD_C. This BDD is constructed as the conjunction of individual BDDs encoding each interval constraint on the state variables, together with an additional BDD capturing the relational constraint between the two speed variables. The property holds if and only if there exists no state satisfying the constraints for which the system issues a non-COC advisory. Formally, this amounts to checking the emptiness of the following intersection: $BDD_C \wedge BDD_{acas} \wedge \neg COC$. If this intersection is empty, the property is validated; otherwise, any satisfying assignment provides a concrete counterexample. The results of this verification are reported below. It illustrates how complex domain-specific and relational operational properties can be expressed and verified efficiently within the BDD framework.

P	$\mathbf{BDD_{COC}}$	$\mathbf{BDD_{WL}}$	$\mathbf{BDD_{WR}}$	$\mathbf{BDD_{SL}}$	$\mathbf{BDD_{SR}}$
$\mathbf{P_{11}}$	Valid	Valid	Valid	Valid	Valid

6 Onboard deployment

6.1 `C` code generation

Using the proposed construction strategy, we obtain five BDDs corresponding to the five ACAS-Xu LUTs used for horizontal collision avoidance, stored in the standard DDDMP format. These DDDMP representations are then parsed to generate an equivalent standalone `C` implementation of the BDD evaluation logic, in which the diagram structure and variable ordering are explicitly encoded without relying on any external libraries. The resulting implementation consists of approximately 1.5 million lines of `C` code encoding the BDD structure, together with a lightweight runtime of a few hundred lines implementing the evaluation procedure. It is intentionally kept simple and excludes any dynamic memory allocation, making it well-suited for formal certification processes (which is beyond the scope of this study). This `C`-based implementation significantly improves embeddability, enabling deployment on a wide range of embedded platforms. Finally, correctness is ensured by exhaustively validating the generated code against all entries of the corresponding LUT, confirming a faithful transfer from the symbolic representation to executable logic.

6.2 Memory footprint

Beyond deployment, our objective is to systematically compare the proposed BDD based compression with NN based approaches. BDDs provide substantial reductions in storage requirements by exploiting structural regularities in the decision logic. In practice, the storage cost of a BDD-based implementation is determined by the size of the generated `C` code required to evaluate the diagram. We measured then the memory footprint of the `C` code compiled with `gcc` and the option `-O3`. This code fully captures the decision logic and can be directly deployed on embedded targets without requiring additional data structures or lookup tables. In comparison, the memory footprint of the neural-network-based implementation can be directly estimated from the network architecture [20]. The raw size of a single network is approximately 53 KB, leading to a total size of about $9 * 53$ KB, i.e., roughly 0.5 MB, to represent one LUT. However, the additional memory required by the Safety Net mechanism, which is necessary to guarantee correctness when neural networks alone are insufficient [13,4], is missing. According to [10], once the Safety Net is included, the overall storage requirement grows to approximately 65 MB in a setting where one LUT parameter is fixed.

Table 2: Memory footprint comparison of ACAS-Xu representations (MB)

	COC	**WL**	**WR**	**SL**	**SR**
LUT	944	944	944	944	944
NN	0.5	0.5	0.5	0.5	0.5
NN + Safety Net	≥ 65	≥ 65	≥ 65	≥ 65	≥ 65
BDD	2.7	1.8	1.8	2.8	2.8

Table 2 highlights the drastic reduction in storage requirements achieved by BDDs compared to the original LUTs, while preserving exact behavior. It is important to note that LUTs store the scores for all four possible actions for each state, whereas BDDs encode the decision logic without explicitly storing all these scores. Thus, to make a fair comparison of memory footprint per action, the effective memory requirement of the LUTs would need to be divided by 10: roughly 94 MB per action, it remains substantially larger than the memory required by BDDs. Although neural networks achieve even smaller raw sizes, the additional Safety Net required to ensure correctness largely offsets this benefit.

6.3 Execution time

Finally, BDD evaluation incurs significantly lower and more predictable execution times than alternative representations [15]. We conducted a comprehensive comparative study of three alternative implementations of the ACAS-Xu horizontal collision avoidance logic: the original LUTs, NN-based approximations, and the proposed BDD-based representation. This study was performed across multiple platforms, ranging from a standard desktop workstation to embedded CPU boards and an FPGA-based implementation, in order to assess execution time and deployability under representative operational constraints.

All reported execution-time measurements follow a common experimental protocol. Each experiment consists of evaluating the corresponding decision logic on 10,000,000 randomly generated encounter states, representing slightly more than 10% of the states contained in a single LUT. Moreover, performance evaluation focuses exclusively on the BDD constructed from the SR LUT, which is the largest among the four retained advisory BDDs. All measurements are therefore conducted in the context of a previous advisory $a_{prev} = SR$, placing the evaluation in a worst-case configuration for the BDD-based approach and providing a conservative assessment of its execution performance.

Desktop evaluation: The LUT-, NN-, and BDD-based systems were executed on a Linux workstation equipped with an Intel Core i7-13800H processor. The BDD-based system corresponds to the standalone `C` code generated from the DDDMP representation, as described in Section 6.1. The NN-based implementation relies on a `C++` program using the NNet inference library [29], which loads the appropriate network stored in the `.nnet` format. Finally, the LUT-based system consists of a `C` program that loads the original ACAS-Xu LUT from its binary representation and performs direct table lookups. This desktop experiment provides a controlled reference point for comparing execution latency and variability across the three representations. The results, reported in Table 3, show that in one hand the BDD-based implementation achieves the lowest average execution time. More importantly, it also exhibits the smallest worst-case execution time, a critical property for safety-critical real-time systems. On the other hand, while the precise timing behavior of LUT-based systems can be complex to characterize, we suppose that the challenges are likely driven by the inherent latency of retrieving data from memory structures of such significant volume.

Table 3: Execution time on desktop (μs)

	$\mathbf{t_{min}}$	$\mathbf{t_{max}}$	$\mathbf{t_{mean}}$
LUT	0.970	7963.917	123.348
NN	0.009	3010.368	1.584
BDD	0.087	2640.453	0.453

Embedded CPU deployment: For both the BDD- and NN-based systems, the same compilation and execution pipelines were reused across a range of embedded CPU platforms to evaluate the execution performance. Experiments were conducted on several embedded boards covering diverse architectures and operating environments, including the BeagleBone Green [5], the BeagleBone X15 [6], a RISC-V development board based on the StarFive JH7110 [30], and the Kria KR260 platform [1]. Due to their prohibitive memory footprint, the original ACAS-Xu LUTs could not be loaded on these embedded systems and were therefore excluded from this stage of the evaluation. Nevertheless, the comparison between BDDs and neural networks across heterogeneous embedded CPUs

provides a clear indication of their relative execution performance, predictability, and scalability. The corresponding results are summarized in Tables 4, Tables 5, Tables 6 and Tables 7.

Execution time on embedded platforms (μs)

Table 4: BEAGLEBONE Green

	$\mathbf{t_{min}}$	$\mathbf{t_{max}}$	$\mathbf{t_{mean}}$
NN	1.125	16 358.000	514.137
BDD	3.125	5785.916	7.384

Table 5: BEAGLEBONE X-15

	$\mathbf{t_{min}}$	$\mathbf{t_{max}}$	$\mathbf{t_{mean}}$
NN	0.487	5267.309	71.611
BDD	1.626	1064.002	2.860

Table 6: RISC-V

	$\mathbf{t_{min}}$	$\mathbf{t_{max}}$	$\mathbf{t_{mean}}$
NN	0.500	4574.000	278.890
BDD	3.000	404.250	5.396

Table 7: KRIA KR260

	$\mathbf{t_{min}}$	$\mathbf{t_{max}}$	$\mathbf{t_{mean}}$
NN	0.090	1400.424	198.899
BDD	1.690	188.032	3.136

Across all tested boards, BDD evaluation exhibits consistently low and predictable execution times, on the order of a few microseconds per query. In contrast, neural-network execution times are one to two orders of magnitude larger on average and exhibit significantly higher variability, reflecting both computational cost and memory-access overheads.

6.4 FPGA deployment

Results obtained from the onboard deployment of BDD on embedded CPU-based boards indicate that the execution times are generally low and compatible with integration into high-rate real-time systems. The worst-case execution time can remain significantly higher than the global average. To reduce the CPU load and increase our BDD evaluation logic efficiency, we investigated the acceleration of BDD on a FPGA-based Xilinx ZCU102 MPSoC architecture [31]. This board contains a large FPGA device with up to 32 Mb of Block Random-Access Memory (BRAM, on-chip memory), separated in 912 blocks of 36 Kb each. The FPGA also includes a lot of Digital Signal Processors (DSPs) and Configurable Logic Blocks (CLBs).

Using Vitis High-Level Synthesis (HLS) [2], our generated `C` code (detailed in Section 6.1) was adapted to meet Xilinx design constraints, in particular the AXI (Advanced eXtensible Interface) register-based interface. A custom BDD IP core was subsequently generated and integrated into the Board Support Package (BSP) of the Xilinx ZCU102 evaluation board. Input and output data were exchanged between a bare-metal `C` application and the BDD IP core via memory-mapped AXI4-Lite registers. The IP core exhibits very low logic utilization, requiring less than 2% of the available CLBs and no DSP blocks, while its memory usage is significantly higher, consuming more than 70% of the available BRAM resources. The timing measurements, including AXI register read and write operations, demonstrate excellent performance, with very low and highly stable

processing times (see Table 8). Overall, these results indicate that, despite the relatively high memory usage inherent to BDD structure that must be considered for deployment, employing an FPGA as a BDD accelerator can be a highly efficient solution for the ACAS-Xu decision logic.

Table 8: Execution time of the BDD IP core on ZCU 102 FPGA (μs)

	$\mathbf{t_{min}}$	$\mathbf{t_{max}}$	$\mathbf{t_{mean}}$
BDD	1.200	2.180	1.408

7 Conclusion

This paper introduced a new approach for compressing the operational ACAS-Xu horizontal collision avoidance logic based on BDDs. Unlike approximation-based techniques, the proposed method provides an exact symbolic encoding of the certified ACAS-Xu LUTs: every discretized encounter state is mapped to the same advisory as in the reference LUTs, without loss of fidelity. Beyond compression, we demonstrated through extensive experimentation that this symbolic representation enables efficient and predictable execution on a wide range of platforms, from desktop CPUs to embedded ARM, RISC-V, and FPGA-based systems. Across all tested configurations, the BDD-based implementation consistently exhibits lower and more stable execution times than both LUT- and NN-based solutions, making it particularly attractive for safety-critical and energy-constrained embedded deployments.

A second major contribution of this work is the use of BDDs as a formal verification interface for ACAS-Xu LUTs. Thanks to the exact correspondence between the LUTs and their BDD encodings, verifying a property on the BDD is equivalent to verifying it directly on the ground-truth decision logic. This provides a practical symbolic framework for reasoning about the full-scale ACAS-Xu behavior without relying on approximations or surrogate models. Within this framework, both the system behavior and domain-specific operational properties can be expressed uniformly as Boolean constraints, reducing verification to conjunction and emptiness checks. Beyond this unification, the simplicity and expressiveness of the BDD-based formalism make it possible to efficiently verify not only interval-based properties, but also more expressive relational properties involving interactions between multiple state variables. This significantly expands the class of properties that can be analyzed compared to NN-based verification approaches, which typically rely on restrictive abstractions and simple box-shaped input domains. As a result, the BDD framework enables domain experts, such as flight dynamics or operational safety specialists, to express more meaningful operational properties that closely reflect real-world aircraft behavior, while remaining amenable to exact and automated verification. At the same time, this verification capability also raises a fundamental modeling question. Several operational properties previously shown to hold on NN approximations are violated by the LUTs themselves, despite the latter being the reference of

the ACAS-Xu system. This discrepancy prompts a deeper discussion on where the ground truth of the system should reside: in the exact decision logic encoded by the LUTs, or in higher-level operational specifications intended to capture physical or operational expectations.

Despite its advantages, the current approach has a clear limitation. The constructed BDDs encode only the final advisory decision and do not preserve the action scores stored in the original LUTs. As a consequence, the proposed representation correctly supports single-intruder scenarios but cannot directly handle multi-intruder encounters, where ACAS-Xu selects advisories based on a minimum-over-costs aggregation. Extending the BDD framework to incorporate cost information remains an open challenge. A straightforward encoding of action scores would however lead to a substantial increase in BDD size, as encoding minimum cost values would require approximately 15 additional bits. In this context, using Multi-Terminal BDDs (MTBDDs) [26] or Algebraic Decision Diagrams (ADDs) [3] could offer a more natural formalism for representing these non-Boolean values. This observation points towards future research directions aimed at developing more compact symbolic representations for quantitative decision logic. A potential direction to address this issue is to further reduce the BDD size by exploiting regions of the LUT where the advisory is constant. Large portions of the ACAS-Xu state space correspond to trivial or unambiguous situations (e.g., COC regions), which could be statically identified and handled separately before BDD evaluation. In BDD terms, this amounts to defining "don't-care sets" over such regions and simplifying the decision function accordingly [11]. Classical BDD techniques such as cofactoring and restriction operators make it possible to eliminate irrelevant parts of the input space [12]. These operations have been extensively studied as a means to reduce decision diagram size and complexity and could enable the storage of additional quantitative information, such as action costs.

Overall, this work shows that BDDs offer a compelling alternative to neural-network-based compression for certified avionics decision systems, combining exactness, embeddability, execution predictability, and formal verifiability within a single unified framework.

References

1. AMD / Xilinx: Kria KR260 Robotics Starter Kit Product Brief (2026), `https://www.amd.com/en/products/system-on-modules/kria/k26/robotics.html`, specifications of the KR260 Robotics Starter Kit based on Zynq UltraScale+ MPSoC XCK26
2. AMD Xilinx: Vitis High-Level Synthesis User Guide (UG1399). Advanced Micro Devices, Inc., 2025.2 english edn. (2025), `https://docs.amd.com/r/en-US/ug1399-vitis-hls`, accessed: 2025-11-20
3. Bahar, R.I., Frohm, E.A., Gaona, C.M., Hachtel, G.D., Macii, E., Pardo, A., Somenzi, F.: Algebraic decision diagrams and their applications. Formal Methods in System Design **10**(2-3), 171–206 (1997). `https://doi.org/10.1023/A:1008699807402`

4. Bak, S., Tran, H.D.: Neural network compression of ACAS-Xu early prototype is unsafe. In: NASA Formal Methods Symposium. Springer (2022). https://doi.org/10.1007/978-3-031-06773-0_3
5. BeagleBoard.org Foundation: BeagleBone Green System Reference Manual (2026), https://www.beagleboard.org/boards/seeedstudio-beaglebone-green, specifications of the ARM Cortex-A8 based BeagleBone Green board
6. BeagleBoard.org Foundation: BeagleBone X15 System Reference Manual (2026), https://git.beagleboard.org/beagleboard/beagleboard-x15/-/blob/master/BeagleBoard-X15_SRM.pdf, specifications of the dual-core ARM Cortex-A15 based BeagleBone X15 board
7. Bryant, R.E.: Graph-based algorithms for boolean function manipulation. IEEE Transactions on Computers **C-35**(8), 677–691 (1986). https://doi.org/10.1109/TC.1986.1676819
8. Christian Szegedy, Wojciech Zaremba, I.S., et al.: Intriguing properties of neural networks. In: International Conference on Learning Representations (ICLR) (2014)
9. Clarke, E.M., Grumberg, O., Peled, D.: Model Checking. MIT Press (1999)
10. Clavière, A.: Safety Verification of Neural Network Based Systems Using Formal Methods. Ph.D. thesis, ISAE-SUPAERO (2023), http://www.theses.fr/2023ESAE0041/document, thèse de doctorat, Toulouse. Réf. 2023ESAE0041
11. Coudert, O., Madre, J.C.: Implicit and incremental computation of primes and essential primes of boolean functions. In: Proceedings of the ACM/IEEE Design Automation Conference (DAC). pp. 36–39 (1991)
12. Coudert, O., Madre, J.C.: A unified framework for the formal verification of sequential circuits. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **11**(10), 1265–1278 (1992)
13. Damour, M., de Grancey, F., Gabreau, C., Gauffriau, A., Ginestet, J.B., Hervieu, A., Huraux, T., Pagetti, C., Ponsolle, L., Clavière, A.: Towards certification of a reduced footprint ACAS-Xu system: A hybrid ML-based solution. In: Computer Safety, Reliability and Security. LNCS, vol. 12852, pp. 31–43. Springer (2021). https://doi.org/10.1007/978-3-030-83903-1_3
14. Doran, R.W.: The gray code. Journal of Universal Computer Science **13**(11), 1573–1597 (2007). https://doi.org/10.3217/jucs-013-11-1573
15. Drechsler, R., Becker, B.: Binary Decision Diagrams: Theory and Implementation. Springer (1998)
16. EUROCAE: ED-275: Minimum operational performance standards for airborne collision avoidance system ACAS-Xu. Tech. rep., EUROCAE (Dec 2020), https://www.eurocae.net/documents-and-projects/documents/ed-275/, licensed to ONERA
17. EUROCONTROL: ACAS guide (2022), https://www.eurocontrol.int/system/acas
18. JohnyF, contributors: dd: Decision diagram package for python. https://github.com/johnyf/dd (2015)
19. Julian, K.D., Kochenderfer, M.J.: Policy compression for aircraft collision avoidance systems. Journal of Guidance, Control, and Dynamics **42**(9), 1969–1982 (2019)
20. Julian, K.D., Kochenderfer, M.J., Owen, M.P.: Deep neural network compression for aircraft collision avoidance systems. Journal of Guidance, Control, and Dynamics **42**(3), 598–608 (2019). https://doi.org/10.2514/1.G003724
21. Julian, K.D., Lopez, J., Brush, J.S., Owen, M.P., Kochenderfer, M.J.: A scalable approach to verifying safety properties of neural networks. In: International Conference on Computer Aided Verification (CAV). Springer (2016)

22. Katz, G., Barrett, C., Dill, D.L., Julian, K., Kochenderfer, M.J.: Reluplex: An efficient SMT solver for verifying deep neural networks. In: Majumdar, R., Kunčak, V. (eds.) Computer Aided Verification. pp. 97–117. Springer (2017). https://doi.org/10.1007/978-3-319-63387-9_5
23. Kochenderfer, M.J., Chryssanthacopoulos, J.P.: Robust airborne collision avoidance through dynamic programming. Tech. Rep. ATC-371, MIT Lincoln Laboratory, Lexington, MA (Jan 2011), https://apps.dtic.mil/sti/pdfs/ADA537522.pdf, prepared for the Federal Aviation Administration
24. Lopez, J.G., Ren, L., Meng, B., Fisher, R., Markham, J., Figard, M., Evans, R., Spoelhof, R., Rubenstahl, M., Edwards, S., Pedan, I., Barrett, C.: Integration and flight test of small uas detect and avoid on a miniaturized avionics platform. In: 2019 IEEE/AIAA 38th Digital Avionics Systems Conference (DASC). pp. 1–5 (2019). https://doi.org/10.1109/DASC43569.2019.9081780
25. McMillan, K.L.: Symbolic Model Checking. Ph.D. thesis, Carnegie Mellon University (1993)
26. M.Fujita, P.C.Mcgeer, J.C.-Y.Yang: Multi-terminal binary decision diagrams: An efficient data structure for matrix representation. In: Formal Methods in System Design 10. pp. 149–169 (1997)
27. Rudell, R.E.: Dynamic variable ordering for ordered binary decision diagrams. In: Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD). pp. 42–47 (1993)
28. Somenzi, F.: Cudd: Cu decision diagram package. Tech. rep., University of Colorado at Boulder (1998)
29. Stanford Intelligent Systems Laboratory: NNet: Documentation and scripts related to the .nnet file format (2025), https://github.com/sisl/NNet
30. StarFive Technology Co., Ltd.: StarFive JH7110 SoC Datasheet (2026), https://sizeof.cat/post/starfive-visionfive-2/files/docs/JH7110_Datasheet.pdf, hardware overview and technical specifications of the JH7110 RISC-V processor
31. Xilinx, Inc.: Zynq ultrascale+ mpsoc data sheet: Overview. Tech. Rep. DS891, Xilinx, Inc. (Feb 2022), https://docs.xilinx.com/r/en-US/ds891-zynq-ultrascale-plus-overview